Fortnite Performance Dashboard

Requirement Analysis — ASP.NET Core MVC with AI-Assisted Coaching for Fortnite

Note: Fortnite is a competitive online multiplayer battle-royale video game (esports title), not a physical/traditional sport. All statistics referenced below — kills, wins, K/D ratio, accuracy — are in-game digital metrics retrieved through the FortniteAPI.io third-party API.

Business Problem → Requirement Analysis → System Design → Database Design → Sprint Planning → Development

1. What is Requirement Analysis?

The process of identifying, understanding, documenting, and validating what an organization needs from a software system — covering functionality, users, business rules, security, performance, and acceptance conditions.

2. Case Study

Fortnite, a competitive online multiplayer battle-royale video game, generates rich match data (eliminations, wins, accuracy, playtime) that players currently have to check across scattered tools with no consistent way to see trends. Goal: a web dashboard that pulls a player's Fortnite stats automatically via the FortniteAPI.io, displays stats/trends, and gives rule-based coaching, while admins manage players and platform data.

3. Stakeholders & Actors

- Player — links a Fortnite account, syncs Fortnite stats, views stats and trends, gets coaching suggestions.
- Administrator — manages players, game modes/categories, and platform-wide stats.
- Fortnite Team/Organization — indirect stakeholder tracking player development.
- Primary actors: Player, Administrator (SQL Server, Chart.js, the FortniteAPI.io client, and the recommendation engine are internal components, not actors).

4. Functional Requirements

- Player: register/login, link Fortnite account, sync Fortnite stats via FortniteAPI.io, view dashboard/stats/history, view coaching recommendations.
- Administrator: login, view players and their stats, manage game modes/categories (e.g., Solo, Duo, Squad), view platform-wide statistics.

5. Non-Functional Requirements

- Response time

- Security
- Availability
- Scalability
- Reliability

6. User Stories & Acceptance Criteria

As a player, I want to link my Fortnite account so that my Fortnite stats are imported automatically and I can track how I am doing over time.

- Player must be logged in and have a linked Fortnite account (Epic username).
- System calls the FortniteAPI.io to fetch Fortnite player stats (eliminations, wins, accuracy, K/D ratio, matches played).
- Win rate is calculated automatically after the sync.
- Stats appear in dashboard and history.

7. Given – When – Then

Given the player has linked their Fortnite account, when the player triggers a stat sync, then the system fetches player data from the FortniteAPI.io, saves it, updates statistics, and updates the dashboard.

8. System Requirements

- Backend: ASP.NET Core MVC (.NET 8), C#, Entity Framework Core.
- External API: FortniteAPI.io (Fortnite player stats and match data).
- Database: Microsoft SQL Server.
- Frontend: HTML, CSS, Bootstrap, JavaScript, Chart.js.
- Server: min. 4GB RAM, dual-core; Client: any device with a modern browser.

9. Database Requirements

Four tables cover the current scope, with no unnecessary entities.

- Users — UserId, Name, Email, Password, Role
- Players — PlayerId, UserId, FortniteUsername, Game, Team
- Stats — StatId, PlayerId, Eliminations, Wins, Accuracy, KDRatio, MatchesPlayed, LastUpdated
- Recommendations — RecommendationId, PlayerId, RecommendationText, CreatedDate

A Player has one Stats record (updated on each sync) and many Recommendations (generated from stats). FortniteUsername links the Player record to the FortniteAPI.io.

10. Module Descriptions

- Authentication & Roles — login/registration, role-based access.
- Fortnite API Integration — fetches player stats using a linked Fortnite username.
- Stats Calculation — K/D ratio, win rate, accuracy trends.
- Dashboard — summary stats and Chart.js visuals.
- AI-Assisted Coaching — rule-based recommendations from stats.
- Admin Panel — manage players, categories, view platform stats.

11. System Workflow

Login → Link Fortnite Account → Sync Stats (FortniteAPI.io) → Validate & Save → Recalculate Trends → Generate Recommendation → Update Dashboard

12. System Architecture

Presentation (Razor/Bootstrap/Chart.js) → Controller → Service Layer (FortniteAPI.io Client) → EF Core → SQL Server

Controllers stay thin; the service layer calls the FortniteAPI.io, handles calculations and recommendations, and is the only layer that talks to EF Core.

13. Constraints & Assumptions

- Semester-length project with a small team.
- Player stats depend on the FortniteAPI.io's availability and rate limits (10 requests/min on free tier).
- Coaching is rule-based only, not a trained AI/ML model.
- Single SQL Server database; web-responsive only, no mobile app.
- Assumes players have linked Fortnite account with public stat visibility.

14. Future Enhancements (Out of Scope)

- Real-time, in-match tracking rather than periodic stat sync.
- Weapon and loadout analytics beyond core K/D stats.
- Trained AI/ML model replacing the rule-based engine.
- External AI/LLM API for richer coaching feedback.

The rule-based engine can later be replaced by a real AI/LLM API without changing the database design.

15. What Comes Next?

Requirement Analysis → System Design → Database Design → Sprint Planning → Development